Assignment 7: Denavit-Hartenberg

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

Due: 11:30 AM EST, November 5, 2026

Contents

1 Overview **3**

2 Background **4**
2.1 Parameterizations of 3D Rotations 4
2.2 Denavit-Hartenberg Parameters 4

3 Instructions **6**

4 Written Questions **7**

5 Code Questions **17**

6 Submission Checklist **20**

1 Overview

This assignment reinforces the following topics:

- Parameterizations of 3D Rotations
- Denavit-Hartenberg Parameters
- Forward Kinematics in 3D

2 Background

2.1 Parameterizations of 3D Rotations

Along with the 3x3 rotation matrices, there exists the Euler angles, Roll-Pitch-Yaw, axis-angle, and quaternion conventions for representing 3D rotations. The quaternion convention is used frequently in robotics because it avoids the Euler angles' issues with gimbal lock, uses only 4 numbers as compared to the 9 in rotation matrices, and is overall more numerically stable and more efficient. The conversion between the axis-angle representation and quaternion representation is shown below:

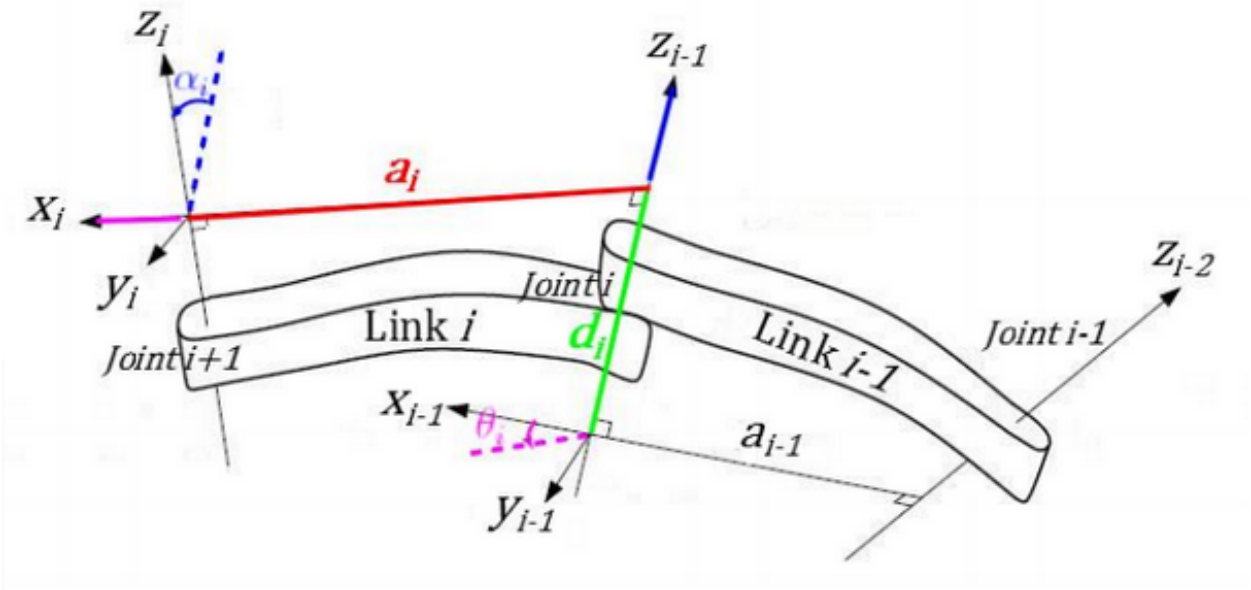
$q_x = a_x * \sin(\frac{\theta}{2})$
$q_y = a_y * \sin(\frac{\theta}{2})$
$q_z = a_z * \sin(\frac{\theta}{2})$
$q_w = \cos(\frac{\theta}{2})$

where:

the axis is normalized so: $a_x^2 + a_y^2 + a_z^2 = 1$
the quaternion is also normalized so $q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1$

2.2 Denavit-Hartenberg Parameters

Recall that the Denavit-Hartenberg convention is a systematic and standard way for selecting reference frames for the links of a kinematic chain. There are 4 Denavit-Hartenberg parameters that are used to represent the transformation between the reference frames. We will briefly illustrate the parameters between again below:



a_i Link length α_i Link twist d_i Link offset θ_i Joint angle

To construct the homogeneous transformation matrix between frames using the 4 D-H parameters, we utilize the following convention:

$$H_i^{i-1} = Rot_{z,\theta_i} Trans_{z,d_i} Trans_{x,a_i} Rot_{x,\alpha_i}$$

$$= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3 Instructions

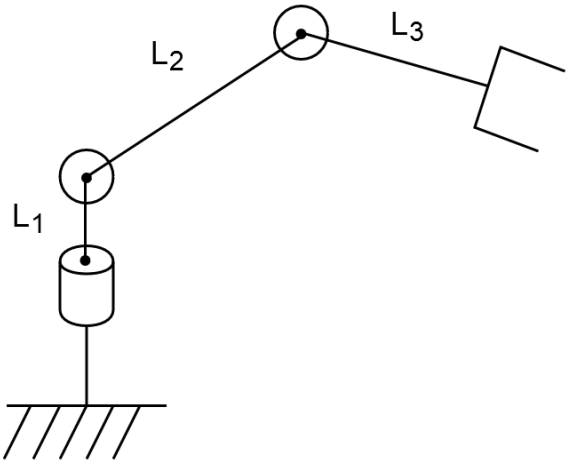
- The due date is on the title page.
- There are two separate submissions to Gradescope: your written answers as a PDF, `<andrew_id>.pdf`, and your code, as the `handin.zip` that `create_submission.py` makes. See the complete submission checklist at the end, to ensure you have everything.
- Each question (for points) is marked with a **points** heading.
- **Start early!** This homework may take a long time to complete.
- **During submission indicate the answer/page correspondence carefully when submitting on Gradescope.** If you skip a written question, just submit a blank page for it. This makes our work much easier to grade.
- If you have any questions or need clarifications, please post in Piazza or visit the TAs during the office hours.
- Unless otherwise specified, **all units are in radians, meters, and seconds**, where appropriate.

4 Written Questions

For all of the following questions, please fully compute all answers and show your work for full credit, unless otherwise specified. Submit your answers as a PDF named `<andrew_id>.pdf`. All answers must be typed, but diagrams may be hand-drawn and scanned in. However, they must be tidy and fully legible! Consider drawing them in a black or blue pen. All units are in radians and meters, where appropriate.

1) Denavit-Hartenberg Practice

Please use the diagram of the arm below for the following question:



(1) [4 points] Label the X and Z axes for each reference frame using the Denavit-Hartenberg convention.

(2) [6 points] Determine the Denavit-Hartenberg Parameters for the robot above.

Link	a_i	α_i	d_i	θ_i
1				
2				
3				
4				

2) Parameterizations of Rotations Practice

Please convert between the different parameterizations of rotations below. **Show all of your work.**

(1) [8 points] Rotation Matrix: $R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$

ZYZ Euler Angles: $\phi = ?, \theta = ?, \psi = ?$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $=$?

Quaternion: $[?, ?, ?, ?]$

(2) [8 points] ZYZ Euler Angles: $\phi = \frac{\pi}{2}$, $\theta = -\frac{\pi}{2}$, $\psi = \pi$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

(3) [8 points] Roll-Pitch-Yaw Angles: $X = 0$, $Y = -\frac{\pi}{2}$, $Z = 0$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

(4) [8 points] Axis-Angle: Axis $= [0, 0, 1]$ Angle $= \pi$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

(5) [8 points] Quaternion: $[0, 0, 0, 1]$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

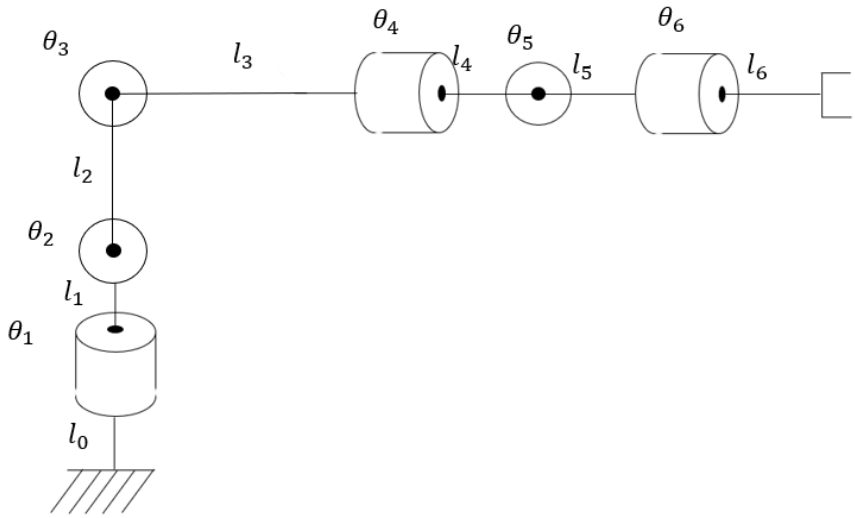


Figure 1: Side View of 6DOF

3) Denavit-Hartenberg Parameters Practice

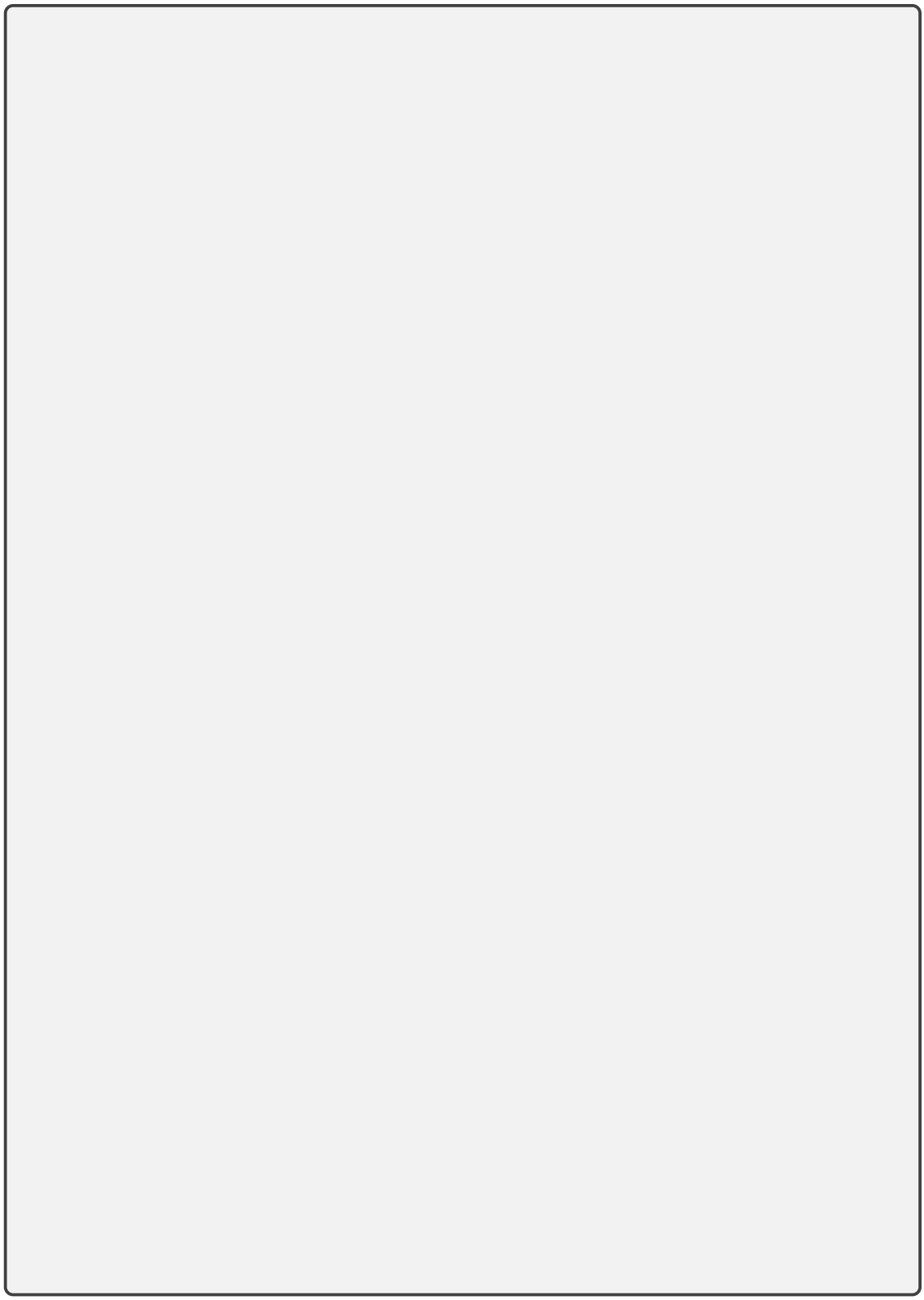
(1) [10 points] Label the X and Z axes for each reference frame using the Denavit-Hartenberg convention.



(2) [20 points] Determine the Denavit-Hartenberg Parameters for the robot.

Link	a_i	α_i	d_i	θ_i
1				
2				
3				
4				
5				
6				

(3) [12 points] Write down the Transformation Matrices between each link using the Denavit-Hartenberg Parameters you determined above.



5 Code Questions

In Assignment 3 you wrote forward kinematics for a planar chain, with 3×3 homogeneous transforms built from link lengths. Here you will do the same for an arm in 3D: the xArm7 from the lab, described by its Denavit-Hartenberg parameters, with 4×4 transforms. The code is in the `ex1` folder of the GitHub repository; run everything from the repository's top folder.

The graded code needs only NumPy (and Matplotlib for the sample plot). The simulator demo at the end needs the course robot library, installed from the copy in the repository, the same way as in Assignment 4:

```
cd 16384-robot-lib
conda env update -f environment.yml
conda activate 16384
cd ..
```

1) Forward Kinematics of the xArm7

Open `ex1/robot.py`. As in Assignment 3, it holds a general `Robot` class for a fixed-base chain of revolute joints, but this one is built from a DH table: an $n \times 4$ array with one row $[a_i, \alpha_i, d_i, \theta_i^{\text{offset}}]$ per joint, in metres and radians. Joint i's DH angle is $\theta_i = q_i + \theta_i^{\text{offset}}$, where q_i is the joint angle the arm reports. The file also holds the xArm7's table, from UFACTORY, as `XARM7_DH`:

Joint i	a_i (m)	α_i	d_i (m)	θ_i^{offset}
1	0	$-\pi/2$	0.267	0
2	0	$\pi/2$	0	0
3	0.0525	$\pi/2$	0.293	0
4	0.0775	$\pi/2$	0	0
5	0	$\pi/2$	0.3425	0
6	0.076	$-\pi/2$	0	0
7	0	0	0.097	0

Figure 2 shows the arm at its zero pose (every $q_i = 0$), with its joint axes. The xArm7 is then

```
robot = Robot(XARM7_DH)
frames = robot.fk(np.zeros(7))
```

Your code has to work for any DH table, though, not only this one. The file walks you through each part in numbered TODOs.

(1) [15 points] Implement `dh_transform(a, alpha, d, theta)`, the transform H_i^{i-1} from frame i to frame $i-1$ for one link, using the convention from the Background section: $H_i^{i-1} = Rot_{z,\theta_i}\, Trans_{z,d_i}\, Trans_{x,a_i}\, Rot_{x,\alpha_i}$. Build the four matrices and multiply them in that order, or write out their product.

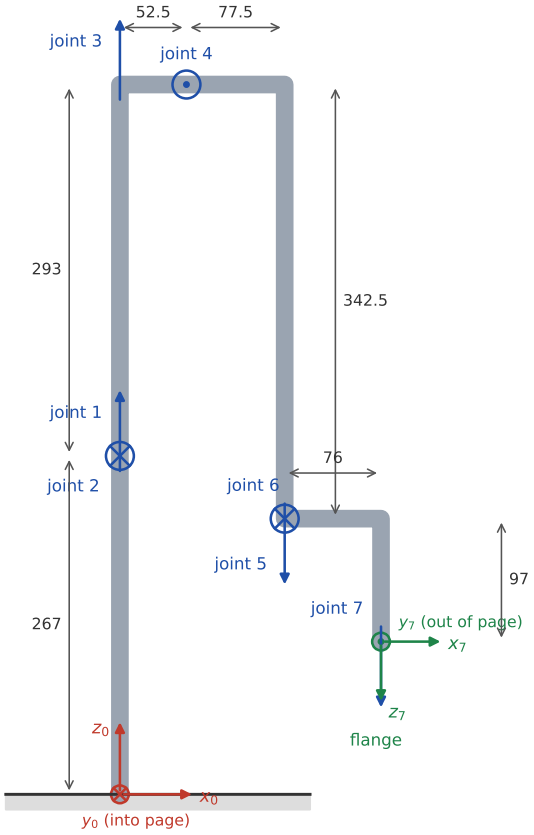


Figure 2: The xArm7 at its zero pose, side on, in mm. The view looks along $+y$, so $\otimes$ is a joint axis along $+y$ (into the page) and $\odot$ one along $-y$. Frame 0 is the base frame, and frame 7 the flange.

(2) [35 points] Implement `forward_kinematics(thetas)`, which returns every DH frame at once: an array `frames` of shape $(4, 4, n+1)$ with `frames[:, :, i]` $= H_i^0$, for $i = 0, 1, \ldots, n$. Frame 0 is the base frame, so `frames[:, :, 0]` is the identity; frame i sits at the end of link i; and the last frame, `frames[:, :, -1]`, is the end effector (on the xArm7, the flange). As in Assignment 3:

- start from frame 0;
- loop over the joints, using H_{i-1}^0 to compute $H_i^0 = H_{i-1}^0 H_i^{i-1}$, with H_i^{i-1} from `dh_transform` and row i of the DH table;
- remember each joint's offset.

Verification and Submission

1. Run the local autograder from the repository's top folder:

   ```
   python local_autograder.py
   ```

It runs your `dh_transform` on 163 links and your `forward_kinematics` on 98 chains (the xArm7 at 42 poses, and random chains of one to seven joints) from `ex1/inK.txt`, compares with `ex1/outK.txt`, and says, for each failure, where in the chain the first mistake is: for part (a), which of the four factors is wrong (it retries your `dh_transform` one parameter at a time), or that they are multiplied in the wrong order; for part (b), which step of the chain is the first wrong one, and usually why: the product order, a forgotten offset, the wrong row or joint angle, or a `dh_transform` result that is itself wrong. Add `-v` to list every failure.

2. Run `python sample_path.py` from the `ex1` folder. It plots the path your forward kinematics gives the xArm7's flange through a sample run, the arm writing RKD in the Assignment 5 lab, next to the correct path.

3. See it on the simulated xArm7:

   ```
   python local_autograder.py -p vis1
   ```

   opens the arm in your browser with a slider per joint. Your `forward_kinematics` draws all eight of its frames over the arm, and the status line checks them against the simulator's own model: your end effector against the flange, and each frame's z axis against the next joint's axis. A Matplotlib window beside it draws the same frames as a stick figure in 3D, with a cross where the simulator's flange really is.

4. The autograder on Gradescope runs the same tests, but perturbs every case's DH parameters and joint angles a little on each run and checks against the reference solution instead of a file. If the local check passes, you should pass on Gradescope too.

5. To submit, run `python create_submission.py` from the repository's top folder. It writes `handin.zip`; upload that to Gradescope.

6 Submission Checklist

- ☐ Upload `<andrew_id>.pdf`, your answers to the written questions, to Gradescope.
- ☐ Run `local_autograder.py` and confirm it reports no failures.
- ☐ Run `create_submission.py` and upload the resulting `handin.zip` to Gradescope.
- ☐ The code is autograded, with no limit on number of submissions within the mandated deadline.